

Project Title

Taxi Data Analysis and Visualization Using Pandas, Matplotlib and Seaborn

1. Project Overview

Taxi services generate a large amount of trip data every day. This data contains useful information about trips, fares, distances, tips, payment methods, pickup locations, and drop-off locations.

However, raw datasets may contain **missing values** and can be difficult to understand directly. Data visualization helps transform this raw information into meaningful charts and patterns.

In this project, the **Seaborn Taxis dataset** is loaded, cleaned, analyzed, and visualized using **Pandas, Matplotlib, and Seaborn**.

The project includes different visualization techniques to understand:

- Fare trends over time
 - Fare by pickup borough
 - Payment method distribution
 - Trip distance distribution
 - Tip distribution
 - Number of trips by borough
 - Relationship between distance and fare
 - Correlation between numerical variables
 - Pairwise relationships between variables
 - Fare distribution by payment method
-

2. Problem Statement

Raw taxi trip data contains valuable information about customer behavior and taxi operations, but missing values and large amounts of data make it difficult to analyze directly.

The objective of this project is to:

1. Load the taxi dataset.
2. Identify and handle missing values.
3. Analyze important numerical and categorical variables.
4. Create different visualizations using Matplotlib and Seaborn.
5. Identify patterns and relationships in taxi trip data.

6. Present the findings in an easy-to-understand manner.

3. Objectives

The main objectives of this project are:

- To load the Seaborn `taxis` dataset.
- To inspect the structure of the dataset.
- To identify missing values.
- To handle missing values using appropriate methods.
- To convert timestamp data into datetime format.
- To analyze fare and distance patterns.
- To analyze customer payment behavior.
- To compare taxi trips across pickup boroughs.
- To study relationships between numerical variables.
- To create professional data visualizations using Python.

4. Technologies and Libraries Used

Technology / Library	Purpose
Python	Programming language
Pandas	Data manipulation and analysis
Matplotlib	Basic data visualization
Seaborn	Statistical data visualization
Jupyter Notebook / Google Colab	Development environment

5. Dataset

The project uses the **Seaborn Taxis dataset**.

The dataset contains information related to taxi trips, including:

Column	Description
<code>pickup</code>	Pickup date and time
<code>dropoff</code>	Drop-off date and time
<code>passengers</code>	Number of passengers

Column	Description
distance	Trip distance
fare	Fare amount
tip	Tip amount
tolls	Toll amount
total	Total trip amount
color	Taxi color
payment	Payment method
pickup_zone	Pickup zone
dropoff_zone	Drop-off zone
pickup_borough	Pickup borough
dropoff_borough	Drop-off borough

6. Task 1 – Loading the Taxis Dataset

Code

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# Load the taxis dataset
df = sns.load_dataset("taxis")

# Display first five rows
df.head()
```

Explanation

The `sns.load_dataset("taxis")` function loads the built-in Seaborn taxi dataset into a Pandas DataFrame named `df`.

`df.head()` displays the first five records of the dataset.

7. Initial Dataset Exploration

Before cleaning the data, the dataset should be inspected.

Check dataset information

```
df.info()
```

Check number of rows and columns

```
df.shape
```

Check column names

```
df.columns
```

Display first few records

```
df.head()
```

These commands help understand the structure and data types of the dataset.

8. Task 2 – Handling Missing Values

Missing values can affect analysis and visualizations. Therefore, they should be identified and handled appropriately.

Check missing values

```
df.isnull().sum()
```

This shows the number of missing values in each column.

Check columns containing missing values

```
missing_values = df.isnull().sum()missing_values[missing_values > 0]
```

8.1 Handling Numerical Missing Values

For numerical columns, missing values can be replaced using the **median**.

Example:

```
numerical_columns = ['distance', 'fare', 'tip', 'tolls', 'total']for col in numerical_columns:    df[col] = df[col].fillna(df[col].median())
```

The median is useful because it is less affected by extreme values or outliers.

8.2 Handling Categorical Missing Values

Categorical columns can be filled using their **mode**, which is the most frequently occurring value.

```
categorical_columns = ['payment', 'pickup_zone', 'dropoff_zone',  
'pickup_borough', 'dropoff_borough']  
for col in categorical_columns:  
    df[col] = df[col].fillna(df[col].mode()[0])
```

8.4 Verify Missing Values

After handling missing values:

```
df.isnull().sum()
```

This confirms whether the required missing values have been handled.

9. Visualization Using Matplotlib / Pandas Plot

9.1 Line Chart – Fare Over Time

Objective

To visualize how taxi fares change over time using the pickup timestamp.

Code

```
df['pickup'] = pd.to_datetime(df['pickup'])  
plt.figure(figsize=(12, 6))  
plt.plot(df['pickup'], df['fare'])  
plt.title('Fare Over Time')  
plt.xlabel('Pickup Time')  
plt.ylabel('Fare')  
plt.xticks(rotation=45)  
plt.tight_layout()  
plt.show()
```

Explanation

- `pd.to_datetime()` converts the pickup column into datetime format.
- `pickup` is used as the X-axis.
- `fare` is used as the Y-axis.
- The line shows how fares vary over time.

Insight

The line chart helps identify changes and fluctuations in taxi fares across the recorded time period.

10. Bar Chart – Total Fare by Pickup Borough

Objective

To compare the total fare generated from each pickup borough.

Code

```
total_fare_by_borough = df.groupby('pickup_borough')['fare'].sum().reset_index()plt.figure(figsize=(10, 6))sns.barplot(x='pickup_borough', y='fare', data=total_fare_by_borough)plt.title('Total Fare by Pickup Borough')plt.xlabel('Pickup Borough')plt.ylabel('Total Fare')plt.xticks(rotation=45, ha='right')plt.tight_layout()plt.show()
```

Explanation

```
df.groupby('pickup_borough')['fare'].sum()
```

Groups the trips according to pickup borough and calculates the total fare for each borough.

Insight

This visualization helps compare the total fare generated by different pickup boroughs.

11. Pie Chart – Trips by Payment Method

Objective

To show the distribution of trips according to payment method.

Code

```
payment_counts = df['payment'].value_counts()plt.figure(figsize=(8, 8))plt.pie(payment_counts.values, labels=payment_counts.index, autopct='%1.1f%%', startangle=90)plt.title('Distribution of Trips by Payment Method')plt.show()
```

Explanation

`value_counts()` counts the number of trips for each payment method.

The pie chart displays each payment method as a separate slice.

Insight

The chart helps understand customer payment preferences.

12. Histogram – Distribution of Distance

Objective

To visualize how trip distances are distributed.

Code

```
plt.figure(figsize=(10, 6))plt.hist(df['distance'], bins=30,
edgecolor='black')plt.title('Distribution of Trip
Distance')plt.xlabel('Distance')plt.ylabel('Number of
Trips')plt.grid(axis='y', alpha=0.3)plt.tight_layout()plt.show()
```

Explanation

- `distance` is plotted on the X-axis.
- Number of trips is plotted on the Y-axis.
- `bins=30` divides the distance values into 30 intervals.

Insight

The histogram helps identify the most common trip-distance ranges and the spread of trip distances.

13. Box Plot – Tip Distribution by Pickup Borough

Objective

To compare tip distributions across different pickup boroughs.

Code

```
plt.figure(figsize=(10, 6))sns.boxplot(x='pickup_borough', y='tip',
data=df)plt.title('Distribution of Tip Amount by Pickup
Borough')plt.xlabel('Pickup Borough')plt.ylabel('Tip
Amount')plt.xticks(rotation=45, ha='right')plt.tight_layout()plt.show()
```

Explanation

- pickup_borough → categorical X-axis.
- tip → numerical Y-axis.
- Each box represents the tip distribution for a borough.

Insight

The box plot helps compare median tips, spread, and potential outliers between pickup boroughs.

14. Visualizations Using Seaborn

14.1 Count Plot – Trips by Pickup Borough

Objective

To visualize the number of taxi trips in each pickup borough.

Code

```
plt.figure(figsize=(10, 6))sns.countplot(x='pickup_borough',
data=df)plt.title('Number of Trips by Pickup Borough')plt.xlabel('Pickup
Borough')plt.ylabel('Number of Trips')plt.xticks(rotation=45,
ha='right')plt.tight_layout()plt.show()
```

Explanation

countplot() counts the number of records in each pickup borough.

Insight

The chart helps identify which pickup boroughs have more or fewer recorded taxi trips.

15. Scatter Plot – Distance vs Fare

Objective

To visualize the relationship between trip distance and fare.

Code

```
plt.figure(figsize=(10, 6))sns.scatterplot(x='distance', y='fare',
hue='pickup_borough', data=df, alpha=0.6)plt.title('Relationship
Between Distance and
Fare')plt.xlabel('Distance')plt.ylabel('Fare')plt.legend(title='Pickup
Borough')plt.tight_layout()plt.show()
```

Explanation

- distance → X-axis.
- fare → Y-axis.
- hue='pickup_borough' → colors points according to pickup borough.
- alpha=0.6 → makes points slightly transparent.

Insight

The scatter plot helps visualize whether longer trips generally correspond to higher fares and allows comparison between pickup boroughs.

16. Heatmap – Correlation Matrix

Objective

To identify relationships between numerical taxi variables.

Variables used:

- distance
- fare
- tip
- tolls
- total

Code

```
columns = ['distance', 'fare', 'tip', 'tolls', 'total']
correlation_matrix = df[columns].corr()
plt.figure(figsize=(8, 6))
sns.heatmap(correlation_matrix, annot=True, cmap='coolwarm',
            fmt='.2f')
plt.title('Correlation Heatmap of Taxi Variables')
plt.tight_layout()
plt.show()
```

Explanation

`.corr()` calculates the correlation between numerical variables.

Correlation values range from **-1 to +1**.

- Close to +1 → strong positive relationship.
- Close to 0 → weak/no linear relationship.
- Close to -1 → strong negative relationship.

Insight

The heatmap makes it easy to identify which numerical variables have stronger relationships with each other.

17. Pair Plot – Pairwise Relationships

Objective

To visualize pairwise relationships between:

- distance
- fare
- tip
- total

The points are colored according to `pickup_zone`.

Code

```
pair_columns = ['distance', 'fare', 'tip', 'total',
                'pickup_zone']
sns.pairplot(df[pair_columns].dropna(),
            hue='pickup_zone')
plt.show()
```

Explanation

`pairplot()` creates multiple plots showing relationships between numerical variables.

`hue='pickup_zone'` assigns different colors based on pickup zone.

Insight

The pair plot helps identify:

- Relationships between distance and fare.
- Relationships between fare and total.
- Relationships between tip and fare.
- Distribution of individual numerical variables.
- Differences between pickup zones.

Note: If the dataset contains many pickup zones, the pair plot may become visually crowded because each zone gets a different color.

18. Violin Plot – Fare Distribution by Payment Method

Objective

To visualize the distribution of fare for each payment method.

Code

```
plt.figure(figsize=(10, 6))sns.violinplot(x='payment', y='fare',
data=df)plt.title('Distribution of Fare by Payment
Method')plt.xlabel('Payment Method')plt.ylabel('Fare')plt.xticks(rotation=45,
ha='right')plt.tight_layout()plt.show()
```

Explanation

- `payment` → categorical X-axis.
- `fare` → numerical Y-axis.
- Each violin represents the fare distribution for a payment method.
- Wider sections indicate greater concentration of observations at those fare values.

Insight

The violin plot allows comparison of the shape, spread, and density of fare values across payment methods.

19. Summary of Visualizations

Visualization	X-axis	Y-axis	Purpose
Line Chart	Pickup Time	Fare	Analyze fare over time
Bar Chart	Pickup Borough	Total Fare	Compare total fares
Pie Chart	Payment Method	Trip Count	Analyze payment distribution
Histogram	Distance	Trip Count	Analyze distance distribution
Box Plot	Pickup Borough	Tip	Compare tip distributions
Count Plot	Pickup Borough	Trip Count	Count trips by borough
Scatter Plot	Distance	Fare	Study distance-fare relationship
Heatmap	Numerical Variables Correlation		Analyze correlations
Pair Plot	Numerical Variables		Analyze pairwise relationships
Violin Plot	Payment Method	Fare	Compare fare distributions

20. Key Findings to Look For

After running the visualizations, the following areas can be analyzed:

Fare

- How fares vary over time.
- Which pickup borough contributes the highest total fare.
- How fare values are distributed across payment methods.

Distance

- Most common trip-distance ranges.
- Relationship between distance and fare.

Tips

- Differences in tip distributions between boroughs.
- Presence of unusually high tip values.

Payment Methods

- Most frequently used payment method.
- Differences in fare distributions between payment methods.

Location

- Number of trips from each pickup borough.
- Differences between pickup zones.

Numerical Relationships

- Correlation between distance, fare, tip, tolls, and total.
 - Pairwise relationships among important numerical variables.
-

21. Conclusion

This project demonstrates how **Python, Pandas, Matplotlib, and Seaborn** can be used to analyze and visualize taxi trip data.

The dataset was first inspected and cleaned by identifying and handling missing values. Different visualization techniques were then applied to understand fare trends, trip distances, payment methods, tips, pickup locations, and relationships between numerical variables.

The combination of **line charts, bar charts, pie charts, histograms, box plots, count plots, scatter plots, heatmaps, pair plots, and violin plots** provides a comprehensive view of the taxi dataset.

This project demonstrates the importance of data visualization in converting raw data into meaningful and interpretable information for data analysis and decision-making.

22. Learning Outcomes

After completing this assignment, I learned how to:

- Load datasets using Seaborn.
- Work with Pandas DataFrames.
- Identify and handle missing values.
- Use mean, median, and mode for data imputation.
- Convert columns into datetime format.
- Create visualizations using Matplotlib.
- Create statistical visualizations using Seaborn.
- Create line, bar, pie, histogram, and box plots.
- Create count, scatter, heatmap, pair, and violin plots.
- Analyze relationships between numerical variables.
- Compare categorical groups using visualizations.
- Interpret charts and extract meaningful insights from data.

Final Project Flow

**Load Data → Explore Data → Check Missing Values → Clean Data → Analyze Data →
Create Visualizations → Interpret Insights → Conclusion**